This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3660245

DVFL: A Lightweight Verifiable Federated Learning System with Dynamic Obfuscation for Resource-Constrained IoT Devices

Pan Wang, Xiaofeng Lu, *Member, IEEE*, Chaoyi Bian, Pietro Lio

***Abstract*—The proliferation of IoT devices necessitates privacy-preserving on-device training, yet existing Federated Learning systems struggle with privacy leakage and unverifiable aggregation under resource constraints. In this paper, we present DVFL, a lightweight verifiable FL system with dynamic obfuscation tailored for IoT. Distinct from traditional FL, DVFL adopts a decoupled personalization strategy: it trains a globally shared feature extractor while keeping the privacy-sensitive classification head strictly local, replacing it with a normalized dummy value during uplink. This design, supported by a 2-of-2 helper module for zero-sum masking, inherently prevents leakage from sensitive layers while enabling personalized adaptation. Furthermore, we introduce a lightweight dual-verification mechanism that allows clients to independently validate aggregation integrity using low-cost hash operations, avoiding prohibitive digital signatures. Experimental results on MNIST, CIFAR-10, UCI_HAR and MIT_BIH show that DVFL reduces Membership Inference Attack AUC to $\approx 50\%$ and thwarts gradient leakage, while achieving personalized accuracy comparable to undefended baselines.**

***Index Terms*—Dynamic Obfuscation, Verifiable, Membership Inference Attack, Gradient Leakage Attack**

I. Introduction

THE rapid proliferation of internet of things (IoT) devices in smart healthcare, industrial automation, intelligent transportation, and smart cities has led to massive volumes of sensitive data generated at the edge of network [1], [2]. In these scenarios, sharing raw sensor data with a centralized cloud is often infeasible due to stringent privacy regulations, limited backhaul bandwidth, and the high risk of data breaches.Federated learning (FL) uses a distributed collaboration mechanism to enable multiple participants to jointly train a global model while maintaining data localization [3]–[5]. In this framework, each participant only needs to upload the trained model parameters (rather than the original data) to the central server for aggregation, theoretically avoiding the direct risk of data privacy leakage [6]. This feature enables FL to demonstrate unique value in high privacy sensitive fields such as medical image analysis, financial risk control systems, and intelligent end user behavior modeling [7], [8]. Although FL provides basic privacy protection through data localization training, recent research shows that its system still faces serious threats of privacy inference attacks, such as membership inference attacks (MIA) [9]. MIA can infer sensitive member information from other participants' training data by analyzing shared model gradients or parameters.

In recent years, significant progress has been made in combating MIA through various innovative technologies, including encryption techniques [10], [11], trusted hardware [12], [13], and perturbation methods [14]–[16]. Xu [11] proposed an FL framework for multi-party computation (MPC) based on functional encryption, which protects local model parameters from inference. However, because of the complex encryption and decryption processes, they often incur high computational costs. In addition, this solution only protects the client and does not protect the server-side model, leaving potential vulnerabilities in other unprotected components. Lebrunt [12] proposed a proxy-based federated learning privacy protection system that resists attribute inference attacks from aggregation servers by mixing model layer updates among participants, significantly reducing the risk of privacy leakage while maintaining model utility. However, due to the high dimensionality of the underlying model, maintaining a tight balance between privacy and computational overhead remains a challenge. The LDP protocol proposed by Chamikara [16] is a local differential privacy federated learning method suitable for industrial environments, which can provide stronger privacy protection in untrusted participant scenarios while maintaining high model performance. Recently, Cédric Boscher [17] proposed a new lightweight federated learning privacy protection method called DINAR, which uses refined privacy protection methods to specifically obfuscate the layers in neural networks that are most prone to leak privacy information, thereby protecting privacy while maintaining model performance and avoiding computational overhead. However, this method only locally obfuscates the selected layers, while the parameters of other layers are still transmitted in plaintext to the server. This asymmetric protection mechanism may pose a potential risk of gradient leakage attack: attackers may use the received unprotected layer parameters, combined with prior knowledge of the model structure, to reconstruct the original training data through optimization methods such as reverse engineering based on deep leakage from gradients (DLG) [18] or improved deep leakage from gradients (IDLG) [19].

A complementary but equally critical challenge in IoT FL is **verifiable aggregation**. In real deployments, the aggregation server may be selfish or even malicious, returning low-quality

•Pan Wang is with Beijing University of Posts and Telecommunications, Beijing 102206, China. Email: wangpan@bupt.edu.cn.
•Xiaofeng Lu is with Beijing University of Posts and Telecommunications, Beijing 102206, China. Email: luxf@bupt.edu.cn.
•Chaoyi Bian is with Venustech Information Technology Group Co., Ltd., Beijing 100193, China. Email: bian_chaoyi@venusgroup.com.cn.
•Pietro Lio is with the Computer Laboratory, University of Cambridge, Cambridge CB21TN, UK. Email: pl219@cam.ac.uk
Copyright (c) 2025 IEEE. Personal use of this material is permitted. However, permission to use this material for any other purposes must be obtained from the IEEE by sending a request to pubs-permissions@ieee.org.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:53:10 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3660245

or tampered global models to reduce computational cost or bias the training outcome. This problem is exacerbated in multi-stakeholder IoT ecosystems, where each participant must be able to *independently* check that its contributions were correctly incorporated and that the global model has not been forged. Recent work [20]–[23] clearly points out that in distributed model training, both gradient privacy and the integrity of the aggregation result should be emphasized. Byzantine or selfish servers may output low-quality (such as oversimplified) or maliciously forged global model updates to save costs or mislead training, causing clients to receive invalid information and disrupting model convergence and performance. Ensuring that the client can verify the correctness of the aggregated results is an indispensable step in building a robust and trustworthy training system. Xu [20] proposed a dual mask protocol and a homomorphic hash function to ensure the verifiability of the results. The FVFL scheme proposed by Wang [21] ensures the security of client data in federated learning through Paillier encryption and secret sharing technology, and supports flexible participation. In addition, by combining super-incremental sequences to reduce communication overhead and utilizing Lagrange interpolation polynomials to implement verification mechanisms, we ensure the authenticity and reliability of cloud aggregation results. However, although the above methods provide good protection for local model parameters, they lack protection for the global model. Attackers on the client side can still use the global model parameters obtained, combined with tools such as generative adversarial networks, to reconstruct the original data distribution of other clients and implement membership inference attacks.

These observations motivate the following question:How can we design a federated learning system tailored to IoT environments that (i) defends against membership inference and gradient leakage attacks from both servers and clients, (ii) protects both local and global models, and (iii) enables end-devices to verify aggregation correctness—while keeping computation and communication overhead low enough for resource-constrained IoT devices?

To address this challenge, we propose **DVFL**, a lightweight **Dynamic Obfuscation Dual-Verification Federated Learning** system designed for IoT-oriented deployments. DVFL introduces a **2-of-2 MPC helper module** that replaces a monolithic trusted third party with two independent, non-colluding helper nodes operating only on additive secret shares. Helpers collaborate with IoT clients to generate per-round randomness and **zero-sum masks** required for secure aggregation, ensuring that no single entity ever learns complete randomness or mask values. Building on this, DVFL incorporates a **fine-grained dynamic obfuscation** mechanism: in each training round, a global random index selects one privacy-sensitive layer, which is replaced on every client by a deterministic obfuscation value derived from its local randomness and the global random sum. The obfuscated layer is normalized during aggregation so that it does not affect the correctness of the global model, while its true value is retained locally as a *private personalization layer* in the IoT device.

To protect the remaining layers, DVFL applies **zero-sum additive masks** to all non-obfuscated parameters before upload. These masks cancel out in the aggregate, allowing the server to recover the correct shared part of the global model while preventing it from isolating any single client's contribution. On top of this, DVFL designs a **dual-verification mechanism**: each client constructs a lightweight verification value from its randomness, mask, and shared parameters, and the server aggregates these verification values alongside the masked model updates. Upon receiving the aggregated model and verification result, every IoT client can independently check a simple hash-and-addition equation to detect aggregation tampering. Unlike heavy-weight cryptographic solutions (e.g., digital signatures or ZK-SNARKs) that offer unconditional integrity but drain IoT batteries, DVFL prioritizes efficiency. We explicitly accept a dependency on the secrecy of the global random seed—assuming a non-colluding server—to achieve verifiable aggregation with negligible computational cost. This design choice is tailored for massive IoT deployments where standard Public Key Infrastructure verification is often infeasible.

The main contributions of this paper are summarized as follows:

- **IoT-oriented secure FL architecture:** We design a dual-helper federated learning architecture in which two semi-trusted helper nodes cooperate with a central server and clients to generate per-round randomness and zero-sum masks, eliminating a single trusted third party while preserving a simple, deployable system structure.
- **Lightweight dynamic obfuscation for privacy-sensitive layers:** We propose a fine-grained dynamic obfuscation strategy that selectively replaces high-risk layers with deterministic dummy values. This approach prevents the server and other clients from accessing the raw parameters of sensitive layers, effectively mitigating membership inference and gradient leakage attacks. By treating the obfuscated layer as a local personalization module, the system maintains a high local model accuracy while securing the global training process.
- **Dual-verification mechanism with low cryptographic overhead:** We introduce a dual-verification scheme in which the server verifies client-side consistency and each IoT client can independently verify server-side aggregation using only hash and addition operations, enabling practical verifiability without expensive public-key cryptography.

II. System Model and Threat Model

A. IoT-Oriented Federated Learning Architecture

We consider an IoT-oriented federated learning system composed of four types of entities.

- **IoT clients:** $\mathcal{C} = \{C_1, C_2, \ldots, C_N\}$: resource-constrained IoT devices (e.g., sensors, cameras, wearables, industrial controllers) that locally store sensitive data and participate in training. Each client C_i has a private dataset D_i and maintains a local copy of the global model.
- **Aggregation server:** S: A cloud or edge server responsible for orchestrating training rounds, selecting participating

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:53:10 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3660245

clients, collecting their (obfuscated and masked) updates, and computing the global model aggregation.

- **Helper nodes:** H_A and H_B: Two semi-trusted helper nodes are deployed on independent edge gateways or virtual machines managed by different administrative domains. To prevent collusion, a periodic rotation mechanism is employed for these nodes. For example, H_A can be hosted by a trusted key management authority or an independent auditing organization, while H_B is hosted by a commercial cloud service provider. These entities are bound by strict service level agreements and legal data privacy regulations. They do not hold any training data and operate only on additive shares and random seeds to assist with randomness generation, zero-sum mask construction, and verification value computation.

The system proceeds in **synchronous training rounds** indexed by $t = 0, 1, \ldots, T$. At the beginning of the round t, the server S selects a subset of available IoT clients $\mathcal{C}_t \subseteq \mathcal{C}$ according to a scheduling policy. The goal of federated learning is to train a global model w. While standard FL typically minimizes a weighted aggregate loss based on dataset size $|D_i|$, such a weighted aggregation complicates lightweight homomorphic verification.

$$\min_{w} \sum_{C_i \in \mathcal{C}_t} \frac{1}{|\mathcal{C}_t|} \mathcal{L}(w; D_i), \tag{1}$$

B. Threat Model

We adopt a **semi-honest but curious** baseline for most system entities and then consider a set of stronger adversaries. Our threat model is tailored to IoT scenarios where different administrative domains may not fully trust each other.

1) Adversary Types: We consider the following adversaries:

- **Rational and Covertly Adversarial Server:** We consider a rational and covertly adversarial server S that attempts to manipulate the global model or reduce costs without detection. Crucially, we assume S prioritizes reputation and thus avoids high-risk collusion with clients to recover the ephemeral global random seed.
- **Curious or misconfigured helper**: H_A or H_B may be compromised or misconfigured. A compromised helper sees all of its own seeds and shares, may collude with S, but never learns the other helper's shares.
- **Semi-honest IoT clients:**: Some clients follow the prescribed protocol but may be curious about the data of other participants. They may analyze the global model trajectory, their local view, and any auxiliary information to infer additional private information about others. We focus on their impact on privacy leakage and verification rather than on arbitrary poisoning attacks.

2) Adversary Goals: We focus on confidentiality and verifiability.

a) Confidentiality Goals:

- **Local model privacy:** Prevent S or a single helper from accurately reconstructing an honest client's model update or sensitive layer, or from substantially improving MIA success beyond random guessing.
- **Global model privacy:** Prevent semi-honest clients from exploiting the global model trajectory and their local view to infer extra private information about others beyond what is inherently revealed by the final model.

b) Integrity and Verifiability Goals:

- **Aggregation integrity**: Detect when S outputs a global model inconsistent with the actual received masked updates (e.g., dropping honest updates, injecting fake ones, or scaling updates to bias aggregation).
- **Bidirectional verifiability**: Enable lightweight checks where S can verify the basic consistency of client contributions, and each client can verify that the aggregated model is compatible with its own update and the specified aggregation rule.

3) Security Assumptions: To keep the problem tractable in IoT settings, we assume:

- **Non-colluding helpers**: H_A and H_B do not collude. Each may collaborate with S and some clients, but at least one helper remains honest and keeps its shares secret, reflecting the deployment between distinct organizations or domains.
- **non-colluding clients**: In each round, clients in $\mathcal{C}_t$ are honest and do not collude with S or any compromised helper, preventing trivial reconstruction from zero-sum constraints.
- **Secure channels and standard primitives**: All channels use authenticated encryption, and we rely on standard assumptions for pseudorandom generators and cryptographic hash functions.

Under these assumptions, DVFL aims to reduce information leakage against the above adversaries and provide practical aggregation verifiability, while keeping computation and communication overhead feasible for typical IoT deployments.

C. Notation Summary

For clarity, Table I summarizes the key notation used in this paper.

III. PROPOSED SCHEME

In this section, we present the design of the DVFL in detail.The overall framework diagram is shown in Figure 1. We first summarize the design goals, then give an overview of the protocol workflow, followed by the three core mechanisms: (i) helper-assisted randomness and mask generation, (ii) fine-grained dynamic obfuscation of privacy-sensitive layers, and (iii) zero-sum secure aggregation with dual verification.

A. Design Goals

DVFL is designed to achieve the following goals in the system and threat model described in Section II:

- **Lightweight client-side operations:** IoT clients should only perform inexpensive cryptographic and algebraic operations (pseudorandom generator (PRG) evaluations, hash computations, and vector additions) so that the protocol can run on low-power devices without hardware accelerators.
- **Protection of both local and global models:** The protocol should reduce information leakage from both upload and

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:53:10 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3660245

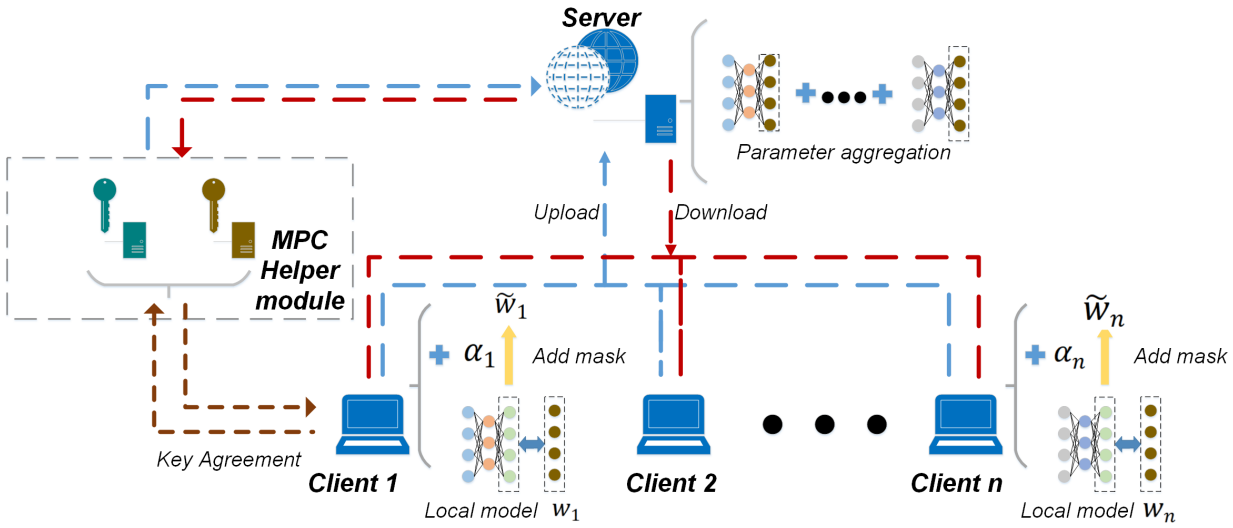


Fig. 1. System framework

TABLE I
NOTATIONS

Notation	Description
$\mathcal{C} = \{C_1, \ldots, C_n\}$	Set of IoT clients
C_i	i-th IoT client
D_i	Private dataset of client C_i
S	Aggregation server
H_A, H_B	Semi-trusted helper nodes
$t \in \{0, 1, \ldots, T\}$	Training round index
$\mathcal{C}_t \subseteq \mathcal{C}$	Selected clients in round t
w^t	Global model parameters at round t
w_i^t	Local model update of C_i in round t
$w_{i,B}^t$	Shared backbone of C_i's local model
$w_{i,\ell^t}^{t,\text{true}}$	True privacy-sensitive layer of C_i
$w_{i,\ell^t}^{t,\text{upload}}$	Obfuscated layer uploaded by C_i
$\hat{w}_i^t$	Obfuscated local update before masking
$\tilde{w}_i^t$	Full masked local update of C_i
$\tilde{w}_{i,B}^t$	Masked backbone update of C_i
$\bar{w}_B^t$	Aggregated backbone at round t
h_i^t	Obfuscation value for C_i's sensitive layer
$\bar{h}^t$	Aggregated obfuscated layer in global model
α_i^t	Zero-sum mask of C_i in round t
$\alpha_i^{t,A}, \alpha_i^{t,B}$	Mask shares from H_A and H_B
$K_{i,A}, K_{i,B}$	Long-term AKE keys between C_i and H_A/H_B
$s_i^{t,A}, s_i^{t,B}$	Per-round seed shares for C_i
s_i^t	Local random seed of C_i in round t
$\rho_i^{t,A}, \rho_i^{t,B}$	Local random vectors from each helper
ρ_i^t	Local random vector of C_i in round t
P^t	Global random sum derived by helpers
P_A^t, P_B^t	Partial random sums held by H_A and H_B
ℓ^t	Random index of privacy-sensitive layer in round t
v_i^t	Verification value computed by C_i
Φ^t	Aggregated verification value at server
r^t	Public random projection derived from P^t
λ	Cryptographic security parameter
q	Modulus of parameter space $\mathbb{Z}_q$

download , mitigating membership inference and deep leakage from gradients attacks from curious servers and semi-honest clients.

- **Verifiable aggregation with low overhead:** Clients must be able to verify that the aggregation server has correctly combined their masked updates according to the specified rule, using only lightweight operations and without resorting to public-key cryptography.
- **No single trusted third party:** Security should not hinge on a single trusted entity. Instead, DVFL uses two non-colluding helper nodes operating on additive shares to replace a monolithic trusted third party.
- **Compatibility with existing FL frameworks:** The design should be compatible with vanilla FL training pipelines, requiring only minimal modifications to model update and aggregation logic.

B. Protocol Overview

At a high level, DVFL augments each FL round with three additional phases, assisted by two helper nodes H_A and H_B:

1) **Helper-assisted randomness setup:** For each selected client C_i in round t, helpers jointly generate additive shares of a local random seed and help derive a global random sum P^t. These random values serve as the basis for both dynamic obfuscation and zero-sum masking.
2) **Client-side protection:** Each client uses its local random seed and the global random sum to:
   - compute a **dynamic obfuscation value** h_i^t to replace a privacy-sensitive layer chosen by a global random index;
   - construct a **zero-sum mask** α_i^t for the remaining shared parameters;
   - derive a **verification value** v_i^t as a light-weight function of its randomness, mask, and parameters.

   The client sends only masked updates and verification values to the server.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:53:10 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3660245

3) **Server-side aggregation and verification:** The server aggregates all masked updates, cancels the masks using the zero-sum structure, reconstructs the shared backbone of the global model, and normalizes the obfuscated layer into a dummy constant. At the same time, it aggregates all verification values into a single Φ_t and returns (w^{t+1}, Φ_t) to clients, which locally check a verification equation.

These steps are integrated into standard FL training without changing the underlying optimization algorithm.

C. Helper-Assisted Randomness and Mask Generation

We assume a standard authenticated key agreement (AKE) primitive that, on input of two parties' identities and public keys, outputs a shared high-entropy session key that is indistinguishable from uniform to any probabilistic polynomial-time adversary that does not corrupt one of the parties. The helper establishes a secure transmission channel with each client through AKE and ensures that data are not visible to the server by transmitting them using symmetric encryption.

Let λ be the security parameter. Each client C_i and each MPC helper $\mathsf{H_A}, \mathsf{H_B}$ receive a long-term public/secret key pair $(\mathsf{pk}, \mathsf{sk})$ for the AKE protocol during system initialization. The server S does not participate in key agreement and never learns any of the keys derived in this phase.

During client registration, C_i runs the AKE protocol separately with each helper. Let

$$K_{i,A} \leftarrow \mathsf{AKE}(C_i, \mathsf{H_A}), \qquad K_{i,B} \leftarrow \mathsf{AKE}(C_i, \mathsf{H_B}) \tag{2}$$

denote the resulting long-term symmetric keys shared between C_i and $\mathsf{H_A}$ and between C_i and $\mathsf{H_B}$, respectively. By the security of the AKE primitive, the keys $K_{i,A}$ and $K_{i,B}$ are computationally indistinguishable from uniform random strings of length λ from any adversary that does not corrupt the corresponding party.

Subsequently, these keys are used both (i) to derive the randomness per-round for the DVFL obfuscation mechanism and (ii) to protect the point-to-point channels between C_i and the helpers using a symmetric authenticated encryption scheme.

Let $\mathsf{F} : \{0,1\}^\lambda \times \{0,1\}^* \to \mathbb{Z}_q$ be a pseudorandom function (PRF) and let t denote the global training round index. In each round t, the helpers derive per-round seed shares for client C_i as

$$s_i^{t,A} = \mathsf{F}\big(K_{i,A}, \mathsf{tag} \parallel t\big), \qquad s_i^{t,B} = \mathsf{F}\big(K_{i,B}, \mathsf{tag} \parallel t\big), \tag{3}$$

where tag is a fixed domain-separation string and $\parallel$ denotes concatenation. Helpers either send these seed shares to C_i over the encrypted channels, or C_i recompute them locally using the same PRF, depending on the concrete implementation.

The client processes the two received seeds using a PRG and sums them to obtain ρ_i^t.

$$\rho_i^t = \rho_i^{t,A} + \rho_i^{t,B} = \mathrm{PRG}(s_i^{t,A}) + \mathrm{PRG}(s_i^{t,B}) \in \mathbb{Z}_q, \tag{4}$$

At the same time, $\mathsf{H_A}$ and $\mathsf{H_B}$ will also sum all their respective seeds to obtain $P^{t,A}$ and $P^{t,B}$, then send them to clients.

$$P^{t,A} = \sum_{C_i \in \mathcal{C}_t} \rho_i^{t,A} \bmod q, \tag{5}$$

$$P^{t,B} = \sum_{C_i \in \mathcal{C}_t} \rho_i^{t,B} \bmod q, \tag{6}$$

The client will sum the received $P^{t,A}$ and $P^{t,B}$ to obtain the special value P^t.

$$P^t = P^{t,A} + P^{t,B} \in \mathbb{Z}_q, \tag{7}$$

According to the method described above, $\mathsf{H_A}$ and $\mathsf{H_B}$ generate zero-sum masks.To construct zero-sum masks, the helpers adopt the following strategies:

Each helper samples random mask shares $\alpha_i^{t,A}$, $\alpha_i^{t,B}$ and $\sum_i \alpha_i^{t,A} = 0$, $\sum_i \alpha_i^{t,B} = 0$ and sends $\alpha_i^{t,A}$, $\alpha_i^{t,B}$ to client C_i, which reconstructs:

$$\alpha_i^t = \alpha_i^{t,A} + \alpha_i^{t,B}. \tag{8}$$

In these strategies, no single helper or server ever learns the complete masks or randomness of any client, as long as the two helpers do not collude.

D. Fine-Grained Dynamic Obfuscation of Privacy-Sensitive Layers

We assume that the model can be decomposed into a shared backbone w_B^t and a set of candidate privacy-sensitive layers $w_{i,\ell^t}^{t,\text{true}}$. Typical choices for these layers include the last one or two fully-connected layers, which are known to be more vulnerable to membership inference.

For each selected client C_i and round t:

a) Local training: The client receives w^t from the server, initializes its local model, and performs local training on D_i to obtain updated parameters w_i^t. The updated local model of client C_i can be decomposed for notational clarity as

$$w_i^t = \big(w_{i,B}^t, w_{i,\ell^t}^t\big), \tag{9}$$

b) Layer selection: Using the globally determined index $\ell^t \sim P^t \bmod \mathcal{T}$, the client identifies the layer to be obfuscated in this round, and the true parameter was stored locally.

$$w_{i,\ell^t}^{t,\text{true}} = w_{i,\ell^t}^t, \tag{10}$$

c) Obfuscation value using P^t: Each client C_i has a scalar local randomness $\rho_i^t \in \mathbb{Z}_q$ derived from its seed s_i^t. The helpers compute and broadcast the global sum

$$P^t = \sum_{C_i \in \mathcal{C}_t} \rho_i^t \bmod q, \qquad P^t \neq 0, \tag{11}$$

together with the number of participating clients $|\mathcal{C}_t|$. The client then computes its obfuscation value as

$$h_i^t = \frac{\rho_i^t}{P^t}, \tag{12}$$

By construction,

$$\sum_{C_i \in \mathcal{C}_t} h_i^t = \sum_{C_i \in \mathcal{C}_t} \frac{\rho_i^t}{P^t} = \frac{1}{P^t}\left(\sum_{C_i \in \mathcal{C}_t} \rho_i^t\right) = 1, \tag{13}$$

so that the normalized aggregate of obfuscation values satisfies

$$\bar{h}^t = \frac{1}{|\mathcal{C}_t|} \sum_{C_i \in \mathcal{C}_t} h_i^t = \frac{1}{|\mathcal{C}_t|}, \tag{14}$$

which is a constant determined only by P^t and independent of any single client's private parameters.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:53:10 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3660245

d) Local Replacement and Personalization: The client replaces the true privacy-sensitive layer by the obfuscation value in its upload.

$$w_{i,\ell^t}^{t,\text{upload}} = h_i^t, \tag{15}$$

Clients locally retain the true parameters $w_{i,\ell^t}^{t,\text{true}}$ to ensure training continuity, reloading them in subsequent rounds rather than the global dummy value. This effectively decouples the system, jointly updating the shared backbone while keeping the privacy-sensitive layer strictly personalized.

$$\hat{w}_i^t = (w_{i,B}^t, w_{i,\ell^t,}^{t,\text{upload}}), \tag{16}$$

After aggregation, the server will only see the obfuscation values h_i^t for the chosen layer in each round, and the global model will contain a normalized layer. As a result, neither the server nor other clients can access the true privacy-sensitive parameters of honest IoT devices.

E. Zero-Sum Secure Aggregation of Shared Backbone

For shared backbone parameters $w_{i,B}^t$, DVFL uses additive zero-sum masks to protect individual updates while allowing the server to recover their average.

1) Client-Side Masking and Upload: For each client C_i in round t, the protection process involves two steps:

a) Mask Application: Using the mask α_i^t provided by the helpers, the client computes the masked update:

$$\tilde{w}_i^t = \hat{w}_i^t + \alpha_i^t, \tag{17}$$

b) Protected Upload: The client sends the tuple $(\tilde{w}_i^t, v_i^t)$ to the server, where v_i^t is the verification value described in Section III-F.

2) Server-Side Aggregation: Upon receiving the uploads from all clients $C_i \in \mathcal{C}_t$, the server performs the following aggregation steps:

a) Backbone Aggregation: The server computes the global backbone as the unweighted mean of the masked updates. This linearity is crucial for preserving the zero-sum property of the masks. Since $\sum_{C_i \in \mathcal{C}_t} \alpha_i^t = 0$ by construction, the aggregation is computed as:

$$\overline{w}_B^t = \frac{1}{|\mathcal{C}_t|} \sum_{C_i \in \mathcal{C}_t} \tilde{w}_{i,B}^t = \frac{1}{|\mathcal{C}_t|} \sum_{C_i \in \mathcal{C}_t} w_{i,B}^t. \tag{18}$$

We adopt unweighted averaging to strictly preserve privacy and ensure lightweight $O(N)$ verification. Unlike standard weighted FL, which would require sharing sensitive dataset sizes to verify the sum, our approach is based solely on the client count $|\mathcal{C}_t|$, assuming a relatively balanced distribution in IoT contexts.

b) Obfuscated Layer Aggregation: Simultaneously, the server aggregates the obfuscated layers:

$$\bar{h}^t = \frac{1}{|\mathcal{C}_t|} \sum_{C_i \in \mathcal{C}_t} (h_i^t + \alpha_i^t). \tag{19}$$

This results in a normalized constant independent of any single client's private parameters, by the design of h_i^t.

Finally, the server forms the new global model:

$$w^{t+1} = \left(\bar{w}_B^t, \bar{h}^t\right), \tag{20}$$

and broadcasts it together with the aggregated verification value Φ^t to the clients.

F. Dual-Verification Mechanism

DVFL designs a dual-verification mechanism that allows both the server and the clients to perform lightweight integrity checks.

1) Client-side Verification Value: To ensure the verifiability of the aggregated backbone while maintaining privacy, we first derive a public random projection r^t from the global randomness P^t in each round t using a hash function:

$$r^t = \mathsf{Hash}(P^t) \mod q \in \mathbb{Z}_q, \tag{21}$$

where $\mathsf{Hash}(\cdot)$ is a publicly known hash function, and P^t is a scalar representing global randomness in round t. The output r^t is a scalar value in the finite field $\mathbb{Z}_q$, derived from the hash of P^t.

Each client C_i computes its verification value v_i^t by taking the inner product of r^t and its masked backbone $\tilde{w}_{i,B}^t$:

$$v_i^t = \rho_i^t + \alpha_i^t + r^t \cdot \tilde{w}_{i,B}^t, \tag{22}$$

where $\tilde{w}_{i,B}^t = w_{i,B}^t + \alpha_i^t$ is the zero-sum masked backbone. Since only masked parameters are involved, the server never sees the raw backbone parameters $w_{i,B}^t$.

After collecting the verification values v_i^t from all clients, the server computes the aggregated verification value:

$$\begin{aligned} \Phi^t &= \sum_{C_i \in \mathcal{C}_t} v_i^t \\ &= \sum_{C_i \in \mathcal{C}_t} \left(\rho_i^t + \alpha_i^t + r^t \cdot \tilde{w}_{i,B}^t\right) \\ &= \sum_{C_i \in \mathcal{C}_t} \rho_i^t + \sum_{C_i \in \mathcal{C}_t} \alpha_i^t + r^t \cdot \sum_{C_i \in \mathcal{C}_t} \tilde{w}_{i,B}^t \end{aligned} \tag{23}$$

Using the zero-sum property $\sum_{C_i \in \mathcal{C}_t} \alpha_i^t = 0$, we have the following:

$$\sum_{C_i \in \mathcal{C}_t} \tilde{w}_{i,B}^t = \sum_{C_i \in \mathcal{C}_t} \left(w_{i,B}^t + \alpha_i^t\right) = \sum_{C_i \in \mathcal{C}_t} w_{i,B}^t = |\mathcal{C}_t|\, \bar{w}_B^t, \tag{24}$$

where $\bar{w}_B^t$ is the aggregated backbone. Therefore, the aggregated verification value becomes:

$$\Phi^t = \sum_{C_i \in \mathcal{C}_t} \rho_i^t + r^t \cdot |\mathcal{C}_t| \cdot \bar{w}_B^t, \tag{25}$$

2) Server-side Consistency Checks: After aggregation, the server can perform basic consistency checks, such as:

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:53:10 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3660245

a) Compute the normalized aggregation of obfuscation values: The server checks whether the aggregated value $\bar{h}^t$ is equal to $\frac{1}{|\mathcal{C}_t|}$:

$$\bar{h}^t \stackrel{?}{=} \frac{1}{|\mathcal{C}_t|}, \tag{26}$$

If $\bar{h}^t = \frac{1}{|\mathcal{C}_t|}$, the server concludes that all uploaded obfuscation values are valid and that the aggregation process is correct.

If $\bar{h}^t \neq \frac{1}{|\mathcal{C}_t|}$, it indicates that there may have been tampering with the aggregation process (e.g., a malicious client or incorrect updates).

b) Handling verification failure: If the verification fails, the server can:

- Discard the updates for the current round and do not use the global model for this round.
- Fall back to the previous global model to avoid using a potentially tampered model.
- Optionally, report the anomaly to a monitoring entity for further investigation.

3) Client-side Aggregation Verification: Upon receiving w^{t+1} and Φ^t, each client C_i uses its local view and public parameters to verify that the server has correctly aggregated the masked updates. From w^{t+1} the client extracts the aggregated backbone $\bar{w}_B^t$, and it can locally reconstruct the same random value r^t using P^t and t via (21). Given the number of participating clients $|\mathcal{C}^t|$, the client computes the *expected* aggregated verification value as:

$$\widehat{\Phi}^t = P^t + r^t \cdot |\mathcal{C}_t| \cdot \bar{w}_B^t, \tag{27}$$

The client then checks whether

$$\Phi^t \stackrel{?}{=} \widehat{\Phi}^t, \tag{28}$$

Any significant deviation indicates potential tampering by the server (e.g., dropping some clients' updates, injecting fake updates, or altering aggregation weights). This verification involves only hash evaluations, vector additions, and one inner product, making it feasible for resource-constrained IoT devices.

If the verification fails, the client can either discard the received global model for this round or fall back to a local fallback policy, and optionally report the anomaly to a monitoring entity.The dynamic obfuscation algorithm is shown in Algorithm 1.

Algorithm 1 Dynamic Obfuscation Dual-Verification Secure Aggregation

Define security parameter λ and initialize the global model w^0.
for each round $t = 0, 1, \ldots, T$ **do**
  MPC helpers
  $\mathsf{H_A}, \mathsf{H_B}$ generate random shares $s_i^{t,A}, s_i^{t,B}$ for each client i.
  compute $P_t^A = \sum_i \rho_i^{t,A}$ and $P_t^B = \sum_i \rho_i^{t,B}$.
  broadcast P_t^A and P_t^B to client i.
  Helpers generate mask shares $\sum_i \alpha_i^{t,A} = 0, \sum_i \alpha_i^{t,B} = 0$ to client i
  Client
  Computes $\rho_i^t = \mathrm{PRG}(s_i^{t,A}) + \mathrm{PRG}(s_i^{t,B}) \in \mathbb{Z}_q$.
  Reconstructs $\alpha_i^t = \alpha_i^{t,A} + \alpha_i^{t,B}$. $P^t = P^{t,A} + P^{t,B}$
  Client trains locally to obtain w_i^t, split into $(w_{i,B}^t, w_{i,\ell^t}^t)$.
  Compute obfuscation index $\ell^t \sim P^t \bmod \mathcal{T}$.
  Store true parameter $w_{i,\ell^t}^{t,\text{true}} = w_{i,\ell^t}^t$.
  Compute $h_i^t = \rho_i^t / P_t$ and replace the sensitive layer: $w_{i,j_t}^t \leftarrow h_i^t$.
  Form obfuscated update $\hat{w}_i^t = (w_{i,B}^t, h_i^t)$.
  Add mask: $\tilde{w}_i^t = \hat{w}_i^t + \alpha_i^t \bmod q$.
  Construct verification value: $v_i^t = \rho_i^t + \alpha_i^t + r^t \cdot \tilde{w}_{i,B}^t$.
  Send $(\tilde{w}_i^t, v_i^t)$ to server.
  Server
  Aggregates parameter:$\bar{w}^t = \frac{1}{|\mathcal{C}_t|} \sum_{C_i \in \mathcal{C}_t} \tilde{w}_i^t$
  Verifies parameter $\bar{h}^t = \frac{1}{|\mathcal{C}_t|}$
  Aggregates shared backbone:$\bar{w}_B^t = \frac{1}{|\mathcal{C}_t|} \sum_{C_i \in \mathcal{C}_t} w_{i,B}^t$
  Computes $\Phi^t = \sum_{C_i \in \mathcal{C}_t} v_i^t$ and broadcasts $(\bar{w}_B^t, \Phi^t, |\mathcal{C}_t|)$.
  Client
  computes $\widehat{\Phi}^t = P^t + r^t \cdot |\mathcal{C}_t| \cdot \bar{w}_B^t$.
  if $\widehat{\Phi}^t = \Phi^t$ **then**
    Recover true parameter: $w^{t+1} = (\bar{w}_B^t,\ w_{i,j_t}^{t,\text{true}})$.
  else
    Reject and request retransmission.
  end if
end for

G. Discussion and Design Trade-offs

DVFL offers several tunable parameters that allow system designers to balance security, accuracy, and overhead.

- **Number and position of privacy-sensitive layers:** Choosing deeper layers for obfuscation can improve privacy against MIA and deep leakage from gradients, but may increase personalization effects and reduce global model coherence.
- **Frequency of obfuscation layer rotation:** The global index ℓ^t can be rotated every round or every few rounds. Faster rotation may better hide patterns across rounds, at the cost of slightly more complex bookkeeping.
- **Mask granularity:** Masks can be applied at full parameter granularity or at block granularity, trading off security granularity against memory and computation.
- **Verification compression:** Verification values can be compressed to scalars to reduce communication overhead, at the risk of slightly increased probability of undetected tampering; the compression function and dimension can be chosen to meet desired integrity levels.
- **Initialization for new clients:** New clients initialize the shared backbone using global parameters $\overline{w}_B^t$ but apply random initialization to the privacy-sensitive layer. Thanks to the pre-trained features in the backbone, the selected layer converges rapidly during the initial local training epochs.
- **Sensitivity of the parameters:** We recommend $\mathcal{T} = 2$ as the optimal configuration. Since the deep layer $d3$ is the most vulnerable to MIA, obfuscating it is essential for privacy. However, extending obfuscation to the output layer ($d1$, i.e., $\mathcal{T} = 3$) destabilizes optimization when deep features are also perturbed. By keeping $d1$ shared, $\mathcal{T} = 2$ allows the output layer to serve as a "global anchor" for class alignment, preserving accuracy while securing the sensitive feature extraction path. Given DVFL's use of lightweight symmetric primitives, computational cost scales linearly.λ is the cryptographic security parameter, set to 128 to maximize security against brute-force attacks while adding minimal

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:53:10 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3660245

burden to IoT devices..

IV. THEORETICAL ANALYSIS

In this section, we give a concise analysis of the security and privacy guaranties provided by DVFL. We first reason about the two-helper module that supplies randomness and zero-sum masks, then analyze the privacy of the dynamic obfuscation mechanism, and finally argue the integrity of the dual verification scheme.

A. Security of the Two-Helper Module

DVFL replaces a monolithic trusted third party with two semi-trusted helpers H_A and H_B operating on additive shares. During client registration, each client C_i establishes two long-term symmetric keys $K_{i,A}, K_{i,B}$ with H_A, H_B via an AKE protocol as in (2). In each round t, helpers derive seed shares $s_i^{t,A}, s_i^{t,B}$ through a PRF and expand them to a random value $\rho_i^{t,A}, \rho_i^{t,B}$ using a PRG, which are summed to obtain ρ_i^t and the global random sum P^t as in (4) $-$(7). They also generate additive mask shares $\alpha_i^{t,A}, \alpha_i^{t,B}$ such that $\sum_{C_i \in C_t} \alpha_i^{t,A} = \sum_{C_i \in C_t} \alpha_i^{t,B} = 0$, which are combined into α_i^t in (8). By the security of AKE, PRF, and PRG, the keys $K_{i,A}, K_{i,B}$, seed shares $s_i^{t,A}, s_i^{t,B}$, and random value $\rho_i^{t,A}, \rho_i^{t,B}$ are computationally indistinguishable from uniform to any PPT adversary that does not corrupt the corresponding parties. Hence, for any honest client C_i, its local randomness ρ_i^t and its contribution to P^t are indistinguishable from uniform as long as C_i, H_A and H_B are not all corrupted. For an honest client, a curious server colluding with at most one helper only learns one mask share (e.g.,$\alpha_i^{t,A}$) and never sees $\alpha_i^{t,B}$. Under the assumption that the client and server do not collude, non-colluding clients in C_t, the unknown shares from the honest helper keep the full masks α_i^t and seeds s_i^t hidden behind the zero-sum constraints. The server only observes masked backbones $\widetilde{w}_{i,B}^t = w_{i,B}^t + \alpha_i^t$ and cannot isolate any single client's raw backbone update beyond what is implied by the final global model. If both helpers are fully compromised and collude with the server, the masks and randomness can be reconstructed and the backbone updates become visible. DVFL therefore explicitly relies on the standard *non-colluding two-helper* assumption: H_A and H_B are operated by independent administrative domains, making their joint compromise harder than compromising a single trusted entity, while keeping helper-side computation to PRG/PRF evaluations and linear additions.

B. Privacy of Dynamic Obfuscation

We next analyze how dynamic obfuscation protects privacy-sensitive layers. In round t, the client C_i splits its local model w_i^t into a backbone $w_{i,B}^t$ and a privacy-sensitive layer $w_{i,\ell^t}^{t,\text{true}}$ as in (9) $-$(10). Using its randomness ρ_i^t and the global sum P^t, the client computes an obfuscation value $h_i^t = \rho_i^t / P^t$ as in (12), replaces the sensitive layer by $w_{i,\ell^t}^{t,\text{upload}} = h_i^t$, and keeps $w_{i,\ell^t}^{t,\text{true}}$ locally as a personalization layer.

By construction, h_i^t is a deterministic function of (ρ_i^t, P^t) and does not depend on $w_{i,\ell^t}^{t,\text{true}}$ or on the client's data. Conditioned on (ρ_i^t, P^t), the obfuscation value h_i^t is independent of the true sensitive parameters and can be simulated without access to $w_{i,\ell^t}^{t,\text{true}}$. Therefore, any adversary who only observes $(h_i^t, \widetilde{w}_{i,B}^t)$ does not gain additional information about the original privacy-sensitive layer.

Moreover, the helpers ensure that $\sum_{C_i \in C_t} \rho_i^t = P^t$ and thus $\overline{h}^t = \frac{1}{|C_t|} \sum_{C_i \in C_t} h_i^t = \frac{1}{|C_t|}$,

as in (13)$-$(14). The aggregated obfuscated layer in the global model is therefore a public constant that does not depend on any single client. The server and other clients never see the true sensitive parameters, and the global model component at the obfuscated position carries no client-specific information.

Combining this with the zero-sum masks on the backbone, we obtain that for any client, the server and semi-honest clients observe only (i) a masked backbone, (ii) a dummy local obfuscation value, and (iii) a constant aggregated layer. Hence, attacks that rely on access to the sensitive layer cannot be directly exploited $w_{i,\ell^t}^{t,\text{true}}$ at the local or global level.

C. Integrity of Dual Verification

Finally, we analyze the integrity guaranties of the dual verification mechanism. In each round t, a public scalar r^t is derived from P^t through (21). Each client C_i computes $v_i^t = \rho_i^t + \alpha_i^t + r^t \cdot \widetilde{w}_{i,B}^t$,

where $\widetilde{w}_{i,B}^t = w_{i,B}^t + \alpha_i^t$. The server aggregates $\Phi^t = \sum_{C_i \in C_t} v_i^t$ and returns Φ^t together with the aggregated backbone $\overline{w}_B^t$ to all clients. Each client recomputes $\widehat{\Phi}^t = P^t + r^t \cdot |C_t| \cdot \overline{w}_B^t$ as in (27) and accepts the round only if $\Phi^t = \widehat{\Phi}^t$.

Completeness. If the server aggregates honestly, then using $\sum_{C_i \in C_t} \alpha_i^t = 0$ and the definition of $\overline{w}_B^t$ in (18), we obtain

$$\begin{aligned}\Phi^t &= \sum_{C_i \in C_t} \rho_i^t + \sum_{C_i \in C_t} \alpha_i^t + r^t \cdot \sum_{C_i \in C_t} \widetilde{w}_{i,B}^t \\ &= P^t + r^t \cdot |C_t| \cdot \overline{w}_B^t = \widehat{\Phi}^t,\end{aligned}$$

so all honest clients accept with probability 1.

Soundness. Consider a malicious server that returns a forged backbone $\overline{w}_B^{t,\star}$ and/or a forged verification value $\Phi^{t,\star}$ inconsistent with the received updates. For fixed client contributions, the difference $\Delta^t = \Phi^{t,\star} - \left(P^t + r^t \cdot |C_t| \cdot \overline{w}_B^{t,\star}\right)$ is a non-zero affine function of r^t whenever the aggregation has been tampered with in a non-trivial way. Since r^t is derived from P^t using a cryptographic hash and is unpredictable before the server fixes its forged outputs, the probability that $\Delta^t = 0$ holds accidentally is negligible in the security parameter. Thus, honest clients detect any significant deviation from the prescribed aggregation rule except with negligible probability.

On the server side, simple checks, such as verifying the normalized obfuscated layer $\overline{h}^t = 1/|C_t|$ in (26) further help detect inconsistent client contributions. Overall, the dual verification mechanism achieves bidirectional verifiability: honest parties accept correct aggregations, and malicious aggregations are rejected with overwhelming probability, using only hash evaluations, additions, and inner products over the backbone dimension, which is practical for IoT devices.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:53:10 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3660245

V. Performance Evaluation

A. Experimental Setup

So far, experiments have only been conducted on an NVIDIA 3060 GPU, and energy consumption/delay data from real IoT devices are not yet available. We selected MNIST [24], CIFAR10 [25], UCI_HAR [26] and MIT_BIH [27] as training datasets, and the PyTorch version is 2.4.0. We evaluate the FL privacy protection methods by measuring the attack area under the curve (AUC), model accuracy and several cost-related metrics. For the measurement of accuracy, the neural network model is ResNet18. The number of clients is 10 and the number of federated learning rounds is 100, the learning rate is set to 10^{-3}, the batch size is 64, and each FL client executes 5 local epochs for all datasets per round. For the measurement of attack AUC, the neural network model is ResNet18, the number of clients is 10, the number of federated learning rounds is 50, the learning rate is set to 10^{-3}, the batch size is 32, and each FL client executes 5 local epochs for all datasets per round. For the gradient leakage experiment, we use the LeNet neural network model, CIFAR10 as the dataset, the number of clients is 10, the learning rate is 10^{-3}, and the number of iterations is 300.

B. Comparative experiment

Our evaluation compares DVFL with different defense schemes, including four state of the art solutions and the no-defense scheme. We consider that LDP [16] is a state of the art solution using differential privacy (DP). DINAR [17] is the foundation of our approach and we optimize it on this basis to enhance its privacy and security.We also take into account a flexible and verifiable privacy preserving federated learning scheme FVFL [21], as well as another verifiable defense solution Verifynet [20]. For LDP, we set the privacy budget parameter $\varepsilon = 2.2$ and the probability of privacy leakage $\delta = 10^{-5}$.

C. Accuracy and Attack AUC

According to the experience of previous studies [17], [28], first, we focus on the last 5 layers of the ResNet18 neural network model (fc-out-$d1$, layer4.1-conv2-$d2$, layer4.1-conv1-$d3$, layer4.0-conv2-$d4$, layer4.0-conv1-$d5$, where d represents the number of layers from back to front). Each layer is individually obfuscated, and then its accuracy is measured and compared with the training accuracy of the model without any obfuscation. After that, we conduct MIA on the global model and local model after single-layer obfuscation, and compare them with the attack on the model without obfuscation. Fig.2 shows the accuracy of the obfuscation method on different datasets.

The results show that, when comparing the accuracy of the obfuscating layers $d1$ to $d3$ on all datasets with that without obfuscation, it can be seen that the training accuracy of all models with the obfuscation strategy increases rapidly but finally converges to different stable levels. Among them, the training accuracy of $d1$ is closest to that without defense. After obfuscating $d4$ and $d5$, the accuracy of the model is greatly reduced.

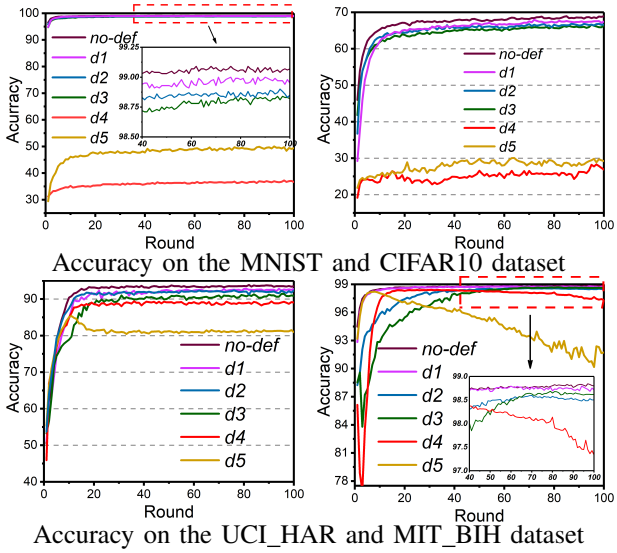


Fig. 2. Accuracy of obfuscation methods on different datasets

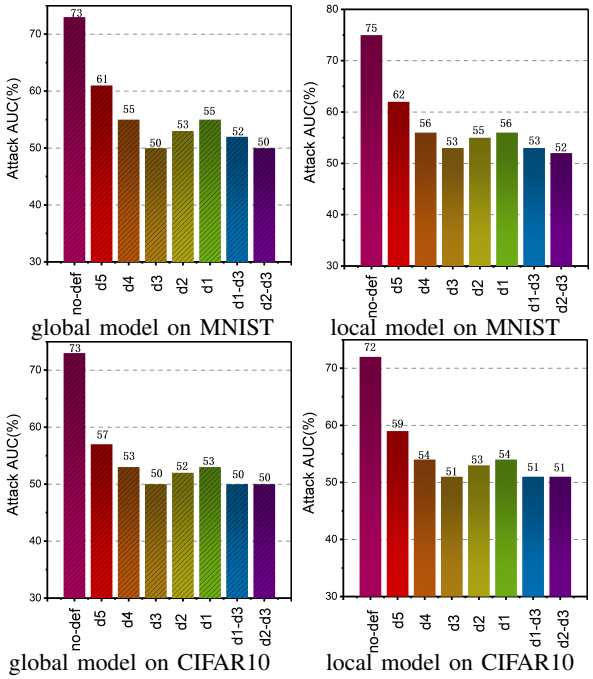


Fig. 3. The attack AUC of obfuscation methods on different datasets

Figs.3 and 4 show the average attack AUC of MIA on the server side and on the client side for different layer obfuscation methods. For the average attack AUC plot, obfuscating the $d3$ layer shows the lowest AUC value, which is 50%. This means that when the server or other clients perform MIA, they can only make guesses. According to the comparison between accuracy and privacy, to balance privacy and accuracy, we select the layer $d1$-$d3$ $\mathcal{T} = 3$ and the $d2$-$d3$ layers $\mathcal{T} = 2$ as random replacement layers for experiments. In Figs.3 and 4, the $d2$-$d3$ obfuscation method shows good performance in resisting MIA attacks for the global model parameters on both datasets, but there is still a certain probability of leaking member information in the local parameter model. However,

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:53:10 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3660245

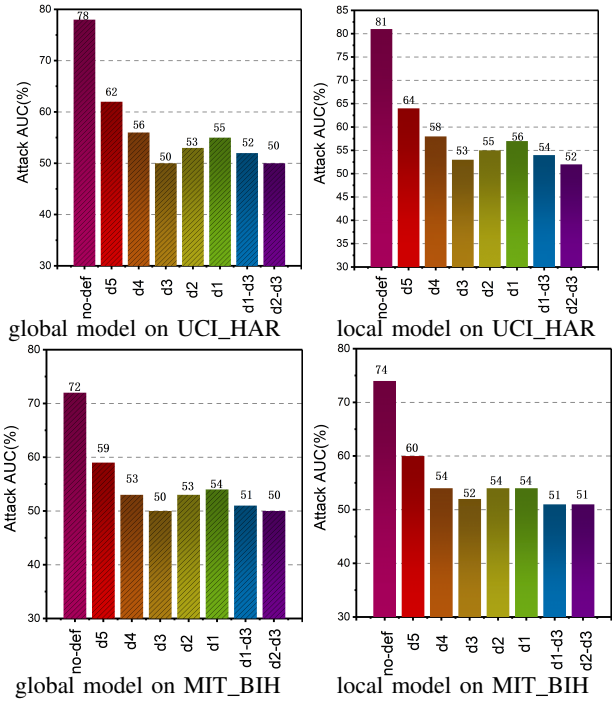


Fig. 4. The attack AUC of obfuscation methods on different datasets

for the $d1$-$d3$ obfuscation method, there is a possibility of leakage for both the global model and the local model on both datasets. Next, we measure the accuracy of these two obfuscation methods.

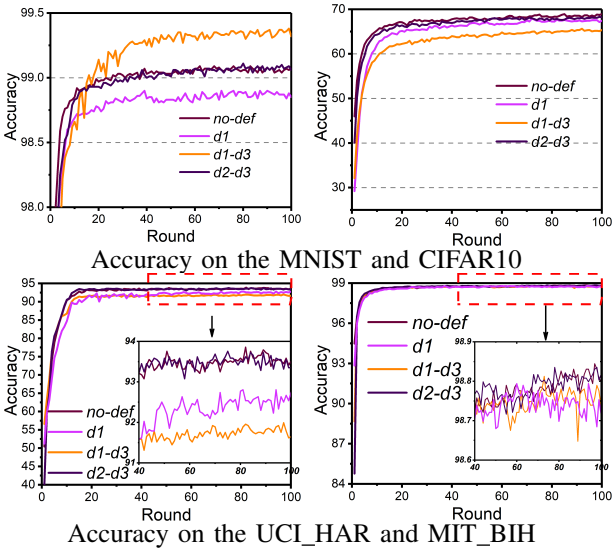


Fig. 5. Accuracy of dynamic obfuscation methods on different datasets

According to the experimental results in Fig.5, on the MNIST dataset, the accuracy of $d1$-$d3$ fluctuates greatly, but the final stability exceeds the accuracy of the undefended one. For $d2$-$d3$ on the MNIST dataset, the performance is relatively smooth, and the final stable value also exceeds the accuracy of the undefended one. For the CIFAR10 and UCI_HAR, the accuracy of $d1$-$d3$ is significantly lower than that of obfuscating $d1$ alone. The performance of $d2$-$d3$ is good and is only slightly lower than the undefended accuracy curve. For MIT_BIH, $d1$-$d3$ and $d2$-$d3$ represents similar accuracy. Considering the attack AUC, the dynamic obfuscation of $d2$-$d3$ has a good balance between accuracy and privacy. For the local attack AUC of the dynamic obfuscation of $d2$-$d3$, it is less than 50%. To solve this problem, we add a zero-sum mask generated by MPC helper module before uploading the local parameters, thus forming our dynamic obfuscation dual verification method. This mask will be offset during the aggregation on the server side, which neither affects the accuracy of the global model nor plays a good role in protecting the local model.

D. Accuracy Comparison

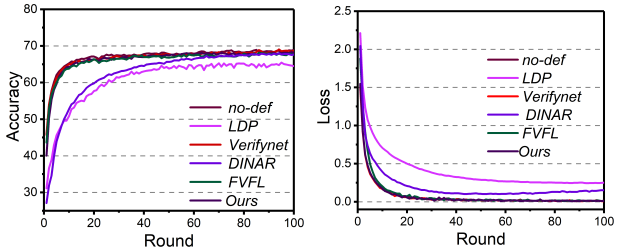


Fig. 6. Comparison of different methods in terms of accuracy and loss on CIFAR10

Fig.6 shows the accuracy measurement of various defense methods on the CIFAR10 dataset. FVFL, Verifynet, and DINAR show approximately the same accuracy as the undefended federated learning framework. FVFL uses the Paillier encryption method to encrypt the model parameters of each client before uploading. After the server aggregates and decrypts them, the real global model parameters will be obtained. The Verifynet method adds a cancelable noise mask to the uploaded local model parameters. After the server aggregates them, the real global model parameters can also be obtained. Therefore, these two methods can maintain accuracy similar to that of undefended federated learning. As shown in Fig.6, verifiable schemes (FVFL, VerifyNet) and our DVFL achieve accuracy comparable to the non-defense baseline. In contrast, LDP incurs a noticeable accuracy drop due to the perturbation noise inherent in differential privacy, highlighting the utility advantage of our obfuscation-based approach. So when the client updates the global model, the model parameters obtained with noise will affect the training accuracy. Our method balances the relationship between security and accuracy through a dynamically adjusted obfuscation strategy, making the accuracy close to that of undefended federated learning.

E. Comparison of Attack AUC

In the following text, we evaluate the effectiveness of DVFL and other protection mechanisms in addressing MIA, i.e. minimizing the attack AUC on global model parameters and client local models.

An attacker runs the MIA described in [29] on different datasets and their underlying models. Fig.7 shows the average

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:53:10 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3660245

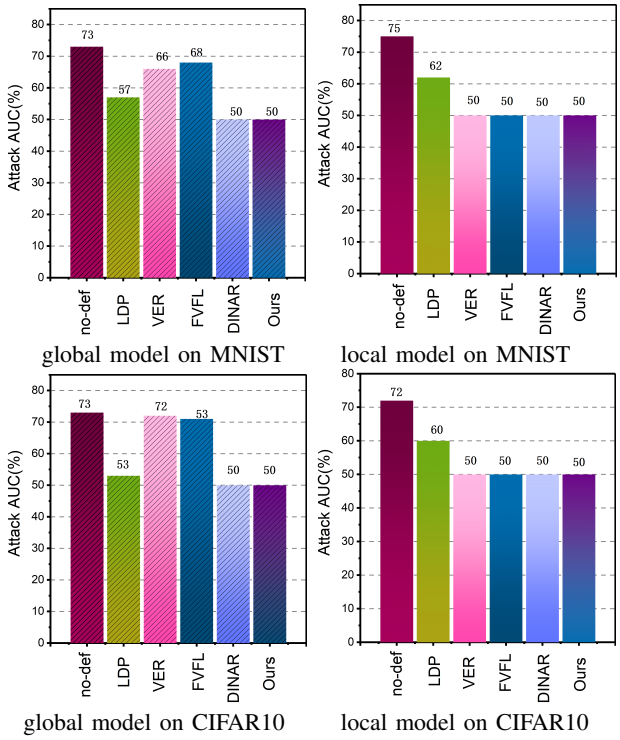


Fig. 7. Comparison of attack AUC on global and local model parameters using different methods

attack AUC for the global model and the local model. The results show that the consistency of the LDP method based on differential privacy is poor and that it cannot effectively mitigate attacks on both the client and the server sides. Since Verifynet and FVFL add masks to the local model parameters, the privacy protection rate on the two datasets for the local model both reaches an attack AUC of 50%. However, since the masks are finally canceled to obtain the real model parameters, attackers on the client side can effectively perform MIA on the real global model. For DINAR, it obfuscates the single-layer model parameters, which prevents the server and other clients from obtaining the real model parameters, effectively protecting against membership inference attacks. For our method, turns the static obfuscation layer into a dynamic one while adding masking to enhance its security against gradient leakage attacks. The final result of the implementation also shows a very good protection effect, that is, the privacy protection rate on all datasets on the server and client sides reaches the optimal value of an attack AUC of 50%.

F. Accuracy Comparison

Fig. 8 evaluates the impact of varying client numbers using the partitioned CIFAR10 dataset. While client-side accuracy negatively correlates with the client count (due to reduced local data size), our method consistently resists MIA, stabilizing the attack AUC at 50% across all settings.

G. Gradient Inversion Image Comparison

We evaluated DLG attacks on LeNet using CIFAR10. As shown in Fig. 9, undefended models allow for quick image reconstruction. Partial defenses (obfuscating layers 2 or 3 layers

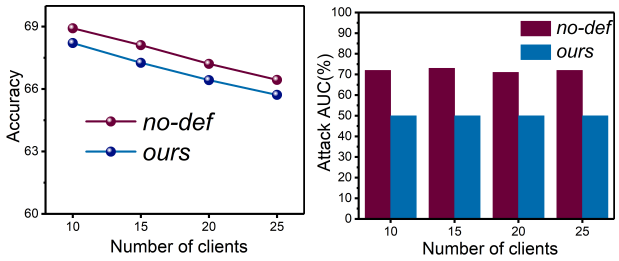


Fig. 8. Comparison of acurracy and attack AUC with different clients

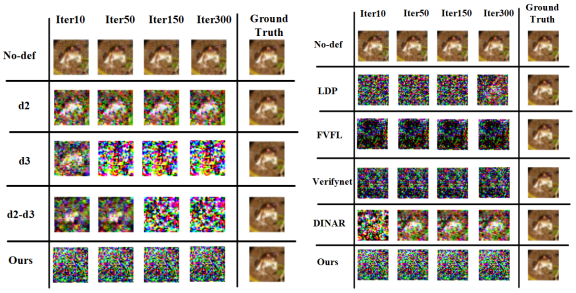


Fig. 9. Gradient leakage attack on local model parameters by different obfuscation layers (left) and different defense method (right)

or DINAR) still yield low-resolution images due to exposed parameters. In contrast, DVFL employs dynamic obfuscation with zero-sum masking to generate indistinguishable noise, effectively preventing leakage similar to LDP, VerifyNet, and FVFL. Table II summarizes that, unlike other methods, DVFL simultaneously resists double attacks and supports verification.

TABLE II
THE DEFENSE PERFORMANCE OF DIFFERENT METHODS

Protection method	MIA	DLG	verifiable
DINAR [17]	✓	×	×
LDP [16]	×	✓	×
FVFL [21]	×	✓	✓
Verifynet [20]	×	✓	✓
Our method	✓	✓	✓

H. The Computation, Communication, and Storage Overhead of Our Method

TABLE III
COMPARISON OF FEDERATED LEARNING METHODS

Method	Device	Computational Complexity	Communication Complexity	Storage Complexity
LDP	Client	O(N)	O(N)	O(N)
	Server	O(N)	O(N)	O(N)
DINAR	Client	O(N)	O(N)	O(N)
	Server	O(N)	O(N)	O(N)
FVFL	Client	O(N² + NV)	O(N + V)	O(N + V)
	Server	O(N²V)	O(N² + NV)	O(N²V)
VerifyNet	Client	O(N² + NV)	O(N + V)	O(N²V)
	Server	O(N²V)	O(N² + NV)	O(N²V)
Our method	Client	O(N)	O(N)	O(N)
	Server	O(N)	O(N)	O(N)
	Helper Node	O(N)	O(N)	O(N)

1) Client-Side Computation: The client-side computation in DVFL includes several tasks:

- **Local Training:** Performs standard model training on the local dataset, with a computational complexity of $O(N)$.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:53:10 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3660245

- **Dynamic Obfuscation:** Computes obfuscation values using local and global randomness. Since this operation is linear regarding the sensitive layer's parameters, the complexity is $O(N)$.
- **Masking and Verification:** Applies additive zero-sum masks and constructs verification values. These steps involve only simple linear arithmetic, maintaining an overall complexity of $O(N)$.

2) Server-Side Computation: The server aggregates the model updates and performs verification:

- **Aggregation:** The server aggregates the masked model updates from all clients. Since the masking is zero-sum, the server can recover the true model parameters. This requires linear aggregation across the number of clients, $O(N)$.
- **Verification:** The server also aggregates verification values computed by the clients and checks the consistency of the aggregated model. This involves simple summation and comparison, which is done in $O(N)$.

3) Helper Node Computation and Communication: The helper nodes in DVFL assist in generating randomness and constructing zero-sum masks:

- **Computational Complexity:** Each helper node performs operations to generate random shares and compute the global random sum. These operations are relatively light and are linear in the number of clients, $O(N)$.
- **Communication Complexity:** Helper nodes communicate with the clients to provide them with random seeds and masks. The communication overhead is also linear, $O(N)$, because each client needs to receive the relevant values.

4) Communication Overhead: In DVFL, communication happens between clients, the server, and the helper nodes:

- **Client to Server:** Each client sends its model update to the server, which includes the masked parameters and a verification value. The communication complexity is linear in the number of parameters, $O(N)$.
- **Server to Client:** The server sends the aggregated model and the aggregated verification value back to the clients. This also has linear communication complexity, $O(N)$.
- **Helper to Client:** The helper nodes send random seeds and masks to each client. The complexity for this is linear as well, $O(N)$.

5) Storage Overhead: The storage overhead in DVFL for each client is modest:

- **Model Parameters:** The client stores the model parameters, which includes both the local model and the sensitive obfuscated layer. This is linear in the number of parameters, $O(N)$.
- **Masks and Randomness:** The client also needs to store the randomness and masks used for obfuscation, which requires additional storage of $O(N)$.

In summary, DVFL achieves strict linear complexity $O(N)$ across computation, communication, and storage dimensions. Compared to verifiable schemes such as VerifyNet and FVFL that rely on homomorphic encryption or heavy cryptographic primitives, DVFL successfully eliminates the additional asymptotic overhead associated with verification. Meanwhile, in contrast to lightweight baselines like LDP and DINAR, DVFL introduces critical bidirectional verifiability without incurring higher algorithmic complexity. This design demonstrates that DVFL provides an optimal balance between security, verifiability, and efficiency, making it uniquely suited to the stringent performance constraints of IoT environments.

VI. Conclusions

This paper presented DVFL, a lightweight verifiable federated learning system for IoT. DVFL employs a decoupled personalization strategy—sharing feature extractors while keeping obfuscated classification heads local—to resolve trade-offs between privacy, integrity, and efficiency. This approach neutralizes Membership Inference Attacks and gradient leakage without compromising utility. Using a 2-of-2 auxiliary module and lightweight hashing, DVFL achieves bidirectional integrity with strict $O(N)$ complexity, avoiding heavy cryptographic overhead. Experiments confirm that DVFL maintains high accuracy while reducing MIA success to $\approx 50\%$. Future work will address current limitations regarding non-collusion assumptions and synchronous aggregation by: (1) developing fully decentralized key management; and (2) extending verification to asynchronous settings for better scalability.

Acknowledgments

This research work was supported by the National Natural Science Foundation of China (Grant No.62572065 and No. 62136006).

References

[1] K. Fizza, P. P. Jayaraman, A. Banerjee, N. Auluck, and R. Ranjan, "Iotqwatch: A novel framework to support the development of quality-aware autonomic iot applications," *IEEE Internet of Things Journal*, vol. 10, no. 20, pp. 17 666–17 679, 2023.

[2] J. Hwang, L. Nkenyereye, N. Sung, J. Kim, and J. Song, "Iot service slicing and task offloading for edge computing," *IEEE Internet of Things Journal*, vol. 8, no. 14, pp. 11 526–11 547, 2021.

[3] Z. Li, S. He, P. Chaturvedi, T.-H. Hoang, M. Ryu, E. A. Huerta, V. Kindratenko, J. Fuhrman, M. Giger, R. Chard, K. Kim, and R. Madduri, "Appflx: Providing privacy-preserving cross-silo federated learning as a service," in *2023 IEEE 19th International Conference on e-Science (e-Science)*, 2023, pp. 1–4.

[4] Q. Mao, L. Wang, Y. Long, L. Han, Z. Wang, and K. Chen, "A blockchain-based framework for federated learning with privacy preservation in power load forecasting," *Knowledge-Based Systems*, vol. 284, p. 111338, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950705123010869

[5] J. Ma, H. Liu, M. Zhang, and Z. Liu, "Vpfl: Enabling verifiability and privacy in federated learning with zero-knowledge proofs," *Knowledge-Based Systems*, vol. 299, p. 112115, 2024.

[6] Z. A. El Houda, D. Nabousli, and G. Kaddoum, "Cost-efficient federated reinforcement learning- based network routing for wireless networks," in *2022 IEEE Future Networks World Forum (FNWF)*, 2022, pp. 243–248.

[7] A. Korkmaz, A. Alhonainy, and P. Rao, "An evaluation of federated learning techniques for secure and privacy-preserving machine learning on medical datasets," in *2022 IEEE Applied Imagery Pattern Recognition Workshop (AIPR)*, 2022, pp. 1–7.

[8] F. C. Orlandi, J. C. S. Dos Anjos, V. R. Q. Leithardt, J. F. De Paz Santana, and C. F. R. Geyer, "Entropy to mitigate non-iid data problem on federated learning for the edge intelligence environment," *IEEE Access*, vol. 11, pp. 78 845–78 857, 2023.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:53:10 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

This article has been accepted for publication in IEEE Internet of Things Journal. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/JIOT.2026.3660245

[9] T. Wang, Q. Yang, K. Zhu, J. Wang, C. Su, and K. Sato, "Lds-fl: Loss differential strategy based federated learning for privacy preserving," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 1015–1030, 2024.

[10] Z. Chen, A. Fu, Y. Zhang, Z. Liu, F. Zeng, and R. H. Deng, "Secure collaborative deep learning against gan attacks in the internet of things," *IEEE Internet of Things Journal*, vol. 8, no. 7, pp. 5839–5849, 2021.

[11] R. Xu, N. Baracaldo, Y. Zhou, A. Anwar, and H. Ludwig, "Hybridalpha: An efficient approach for privacy-preserving federated learning," in *Proceedings of the 12th ACM Workshop on Artificial Intelligence and Security*, ser. AISec'19. New York, NY, USA: Association for Computing Machinery, 2019, p. 13–23. [Online]. Available: https://doi.org/10.1145/3338501.3357371

[12] T. Lebrun, A. Boutet, J. Aalmoes, and A. Baud, "Mixnn: protection of federated learning against inference attacks by mixing neural network layers," in *Proceedings of the 23rd ACM/IFIP International Middleware Conference*, ser. Middleware '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 135–147. [Online]. Available: https://doi.org/10.1145/3528535.3565240

[13] A. A. Messaoud, S. B. Mokhtar, V. Nitu, and V. Schiavoni, "Shielding federated learning systems against inference attacks with arm trustzone," in *Proceedings of the 23rd ACM/IFIP International Middleware Conference*, ser. Middleware '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 335–348. [Online]. Available: https://doi.org/10.1145/3528535.3565255

[14] M. Naseri, J. Hayes, and E. D. Cristofaro, "Local and central differential privacy for robustness and privacy in federated learning," 2022. [Online]. Available: https://arxiv.org/abs/2009.03561

[15] Z. Sun, P. Kairouz, A. T. Suresh, and H. B. McMahan, "Can you really backdoor federated learning?" 2019. [Online]. Available: https://arxiv.org/abs/1911.07963

[16] P. C. Mahawaga Arachchige, D. Liu, S. Camtepe, S. Nepal, M. Grobler, P. Bertok, and I. Khalil, "Local differential privacy for federated learning," in *Computer Security – ESORICS 2022: 27th European Symposium on Research in Computer Security, Copenhagen, Denmark, September 26–30, 2022, Proceedings, Part I*. Berlin, Heidelberg: Springer-Verlag, 2022, p. 195–216. [Online]. Available: https://doi.org/10.1007/978-3-031-17140-6_10

[17] C. Boscher, N. Benarba, F. Elhattab, and S. Bouchenak, "Personalized privacy-preserving federated learning," in *Proceedings of the 25th International Middleware Conference*, ser. Middleware '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 454–466. [Online]. Available: https://doi.org/10.1145/3652892.3700785

[18] L. Zhu, Z. Liu, and S. Han, *Deep leakage from gradients*. Red Hook, NY, USA: Curran Associates Inc., 2019.

[19] B. Zhao, K. R. Mopuri, and H. Bilen, "idlg: Improved deep leakage from gradients," 2020. [Online]. Available: https://arxiv.org/abs/2001.02610

[20] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "Verifynet: Secure and verifiable federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 911–926, 2020.

[21] G. Wang, L. Zhou, Q. Li, X. Yan, X. Liu, and Y. Wu, "Fvfl: A flexible and verifiable privacy-preserving federated learning scheme," *IEEE Internet of Things Journal*, vol. 11, no. 13, pp. 23 268–23 281, 2024.

[22] X. Guo, Z. Liu, J. Li, J. Gao, B. Hou, C. Dong, and T. Baker, "Verifl: Communication-efficient and fast verifiable aggregation for federated learning," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 1736–1751, 2021.

[23] F. Luo, S. Al-Kuwari, and Y. Ding, "Svfl: Efficient secure aggregation and verification for cross-silo federated learning," *IEEE Transactions on Mobile Computing*, vol. 23, no. 1, pp. 850–864, 2024.

[24] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[25] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," Citeseer, Tech. Rep., 2009.

[26] D. Anguita, A. Ghio, L. Oneto, X. Parra, and J. L. Reyes-Ortiz, "A public domain dataset for human activity recognition using smartphones," in *The European Symposium on Artificial Neural Networks*, 2013. [Online]. Available: https://api.semanticscholar.org/CorpusID:6975432

[27] G. B. Moody and R. G. Mark, "The impact of the mit-bih arrhythmia database," *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, no. 3, pp. 45–50, 2001.

[28] M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary, "Federated learning with personalization layers," 2019. [Online]. Available: https://arxiv.org/abs/1912.00818

[29] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," 2017. [Online]. Available: https://arxiv.org/abs/1610.05820



Pan Wang is with Beijing University of Posts and Telecommunications, Beijing 102206, China. He is currently a PhD candidate, focusing on researching privacy protection techniques in machine learning.



Xiaofeng Lu (Member, IEEE) is an Associate Professor at the School of Cyberspace Security, Beijing University of Posts and Telecommunications. He is a distinguished member of China Computer Federation. He received his PhD degree in Beijing University of Aeronautics & Astronautics, Beijing, China, in 2010. During his PhD, he held visiting scholar positions at the Computer Laboratory, University of Cambridge, UK. His main research interests include artificial Intelligence, cybersecurity, privacy computing.



Chaoyi Bian is a Senior Researcher at Venus Tech Information Security Co., Ltd., and a postdoctoral researcher jointly trained at the Venus Tech Postdoctoral Workstation and the Postdoctoral Mobile Station of Beijing University of Posts and Telecommunications. He holds a PhD degree, and his main research interests include big data security and big data security analysis.



Pietro Lio received the M.A. degree from the University of Cambridge, Cambridge, U.K., the Ph.D degree in complex systems and non-linear dynamics from the Department of Engineering, School of Informatics, University of Firenze, Firenze, Italy, and the Ph.D. degree in (theoretical)genetics from the University of Pavia, Pavia, Italy. He is currently a Full Professor of computational biology with the Computer Laboratory, University of Cambridge, Cambridge, and a member of the Artificial Intelligence Group. His research interests include bioinformatics, computational biology modeling, and machine learning to integrate various types of data (molecular and clinical, drugs, social and lifestyle) across different spatial and temporal scales of biological complexity to address personalized and precision medicine.

Authorized licensed use limited to: Khulna Univ of Engineering & Technology - KUET. Downloaded on February 16,2026 at 08:53:10 UTC from IEEE Xplore. Restrictions apply.
© 2026 IEEE. All rights reserved, including rights for text and data mining and training of artificial intelligence and similar technologies. Personal use is permitted,
but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.